

Q1 Second Largest Without Duplicates

```
import java.util.Scanner;

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter size of array: ");

        int n = sc.nextInt();

        int[] arr = new int[n];

        System.out.println("Enter array elements:");

        for (int i = 0; i < n; i++) {

            arr[i] = sc.nextInt();

        }

        int largest = arr[0];

        int secondLargest = -1;

        for (int i = 1; i < n; i++) {

            if (arr[i] > largest) {

                secondLargest = largest;

                largest = arr[i];

            }

            else if (arr[i] != largest && arr[i] > secondLargest) {

                secondLargest = arr[i];

            }

        }

    }

}
```

```

        }
    }

    System.out.println("Second Largest Unique Element: " +
secondLargest);

    sc.close();
}
}

```

Q2 Rotate Array K Steps to the Right

```

import java.util.Scanner;

public class Main {

    public static void reverse(int[] arr, int start, int end) {
        while (start < end) {
            int temp = arr[start];
            arr[start] = arr[end];
            arr[end] = temp;
            start++;
            end--;
        }
    }

    public static void rotateRight(int[] arr, int k) {
        int n = arr.length;
        k = k % n; // Handle k > n
        reverse(arr, 0, n - 1);
        reverse(arr, 0, k - 1);
    }
}

```

```
        reverse(arr, k, n - 1);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter size of array: ");
        int n = sc.nextInt();
        int[] arr = new int[n];
        System.out.println("Enter array elements:");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
        System.out.print("Enter value of k: ");
        int k = sc.nextInt();
        rotateRight(arr, k);
        System.out.print("Rotated Array: ");
        for (int i = 0; i < n; i++) {
            System.out.print(arr[i] + " ");
        }
        sc.close();
    }
}
```

Q3 Valid Anagram Check

```
import java.util.HashMap;

public class Main {

    public static boolean isAnagram(String str1, String str2) {

        str1 = str1.toLowerCase().replace(" ", "");
        str2 = str2.toLowerCase().replace(" ", "");

        if (str1.length() != str2.length()) {
            return false;
        }

        HashMap<Character, Integer> freq = new HashMap<>();

        // Count characters in first string
        for (int i = 0; i < str1.length(); i++) {
            char ch = str1.charAt(i);
            if (freq.containsKey(ch)) {
                freq.put(ch, freq.get(ch) + 1);
            } else {
                freq.put(ch, 1);
            }
        }

        // Match characters from second string
        for (int i = 0; i < str2.length(); i++) {
            char ch = str2.charAt(i);
            if (!freq.containsKey(ch)) {
                return false;
            }
        }
    }
}
```

```

    }
    freq.put(ch, freq.get(ch) - 1);

    if (freq.get(ch) == 0) {
        freq.remove(ch);
    }
}

return freq.isEmpty();
}

public static void main(String[] args) {
    System.out.println(isAnagram("Listen", "Silent"));
    System.out.println(isAnagram("hello world", "world hello"));
    System.out.println(isAnagram("rat", "car"));
}
}

```

Q4 Longest Substring Without Repeating Characters

```

import java.util.HashSet;

public class Main {

    public static int longestSubstring(String str) {
        HashSet<Character> set = new HashSet<>();
        int left = 0;
        int maxLength = 0;
        for (int right = 0; right < str.length(); right++)
        {

```

```
        while (set.contains(str.charAt(right))) {
            set.remove(str.charAt(left));
            left++;
        }
        set.add(str.charAt(right));
        int currentLength = right - left + 1;
        if (currentLength > maxLength) {
            maxLength = currentLength;
        }
    }
    return maxLength;
}

public static void main(String[] args) {
    System.out.println(longestSubstring("abcabcbb"));
    System.out.println(longestSubstring("bbbbbb"));
    System.out.println(longestSubstring("pwwkew"));
}
}
```

Q5 Top Earning Employees per Department

```
SELECT d.dept_name,  
       e.name,  
       e.salary  
FROM Employees e  
JOIN Departments d  
  ON e.dept_id = d.dept_id  
JOIN (  
  SELECT dept_id, MAX(salary) AS max_salary  
  FROM Employees  
  GROUP BY dept_id  
) m  
  ON e.dept_id = m.dept_id  
 AND e.salary = m.max_salary;
```

Q6 Monthly Order Revenue Report

```
SELECT  
  DATE_FORMAT(order_date, '%b') AS month,  
  SUM(amount) AS total_revenue,  
  COUNT(order_id) AS total_orders  
FROM Orders  
WHERE YEAR(order_date) = 2024  
GROUP BY MONTH(order_date), DATE_FORMAT(order_date, '%b')  
ORDER BY total_revenue DESC;
```

Q7 Fetch and Display User Posts

```
async function fetchPosts() {  
  const url =  
  "https://jsonplaceholder.typicode.com/posts?userId=3";  
  
  try {  
    const response = await fetch(url);  
  
    if (!response.ok) {  
      throw new Error(`HTTP Error: ${response.status}`);  
    }  
  
    const posts = await response.json();  
  
    posts.forEach(post => {  
      console.log(post.title.toUpperCase());  
    });  
  
    console.log(`Total posts: ${posts.length}`);  
  } catch (error) {  
    console.log("Failed to fetch posts:", error.message);  
  }  
}  
  
fetchPosts();
```

Q8 Weather Summary with Conditional Logic

```
async function getWeather() {  
  const city = "London";  
  
  const latitude = 51.5074;  
  
  const longitude = -0.1278;
```

```
const url =
  `https://api.open-
meteo.com/v1/forecast?latitude=${latitude}&longitude=${longitude}
&current_weather=true`;

const weatherConditions = {
  0: "Clear sky",
  1: "Mainly clear",
  2: "Partly cloudy",
  3: "Overcast",
  45: "Fog",
  61: "Light rain",
  80: "Rain showers",
  95: "Thunderstorm"
};

try {
  const response = await fetch(url);
  if (!response.ok) {
    throw new Error(`HTTP Error: ${response.status}`);
  }
  const data = await response.json();
  const temperature = data.current_weather.temperature;
  const weatherCode = data.current_weather.weathercode;
  console.log("City    :", city);
```

```
    console.log("Temperature:", temperature + "°C");  
    console.log(  
        "Condition :",  
        weatherConditions[weatherCode] || "Unknown"  
    );  
} catch (error) {  
    console.log("Failed to fetch weather data:", error.message);  
}  
}  
getWeather();
```